

JACKUT

Rede Social de Relacionamentos

Relatório de Design — Milestone 1

Disciplina: Programação 2
Universidade Federal de Alagoas - UFAL
Instituto de Computação - IC
Autora: Laura Beatriz Lins Ramos Mainero
2026

1. Introdução

O Jackut é um sistema de rede social de relacionamentos desenvolvido como projeto acadêmico da disciplina de Programação 2, no Instituto de Computação da Universidade Federal de Alagoas (UFAL). O produto é inspirado no Orkut e permite que usuários criem contas, montem perfis personalizados, adicionem amigos por meio de um sistema de convites e troquem recados entre si.

O objetivo acadêmico do projeto é consolidar os princípios de Programação Orientada a Objetos, com ênfase nos padrões GRASP, SOLID e Clean Code, aplicados em um sistema real e testado por aceitação com a ferramenta EasyAccept.

Este relatório descreve as decisões de design tomadas no Milestone 1, que contempla as User Stories 1 a 4: gestão de contas, edição de perfil, sistema de amizades e troca de recados.

2. User Stories Implementadas

US	Título	Descrição	Status
US1	Gestão de Contas	Criação de conta com login, senha e nome. Autenticação via sessão com ID único gerado por UUID.	✓ Implementada
US2	Edição de Perfil	Criação e edição livre de atributos de perfil: descrição, cidade natal, estado civil, aniversário e outros.	✓ Implementada
US3	Sistema de Amizades	Adição de amigos com convite e aceite bilateral. A amizade só é confirmada quando ambos os usuários se adicionam.	✓ Implementada
US4	Troca de Recados	Envio e leitura de recados entre usuários em fila FIFO por destinatário. Persistência garantida entre execuções.	✓ Implementada

Cada User Story possui dois scripts de teste EasyAccept: o arquivo _1 executa os cenários principais e o arquivo _2 valida a persistência dos dados após encerramento e reinicialização do sistema.

3. Arquitetura do Sistema

3.1 Visão Geral

O sistema adota o padrão arquitetural Facade em conjunto com o conceito de Rich Domain Model (Modelo de Domínio Rico). A arquitetura é organizada em camadas com responsabilidades bem delimitadas:

- Facade — porta de entrada única do sistema, sem lógica de negócio;

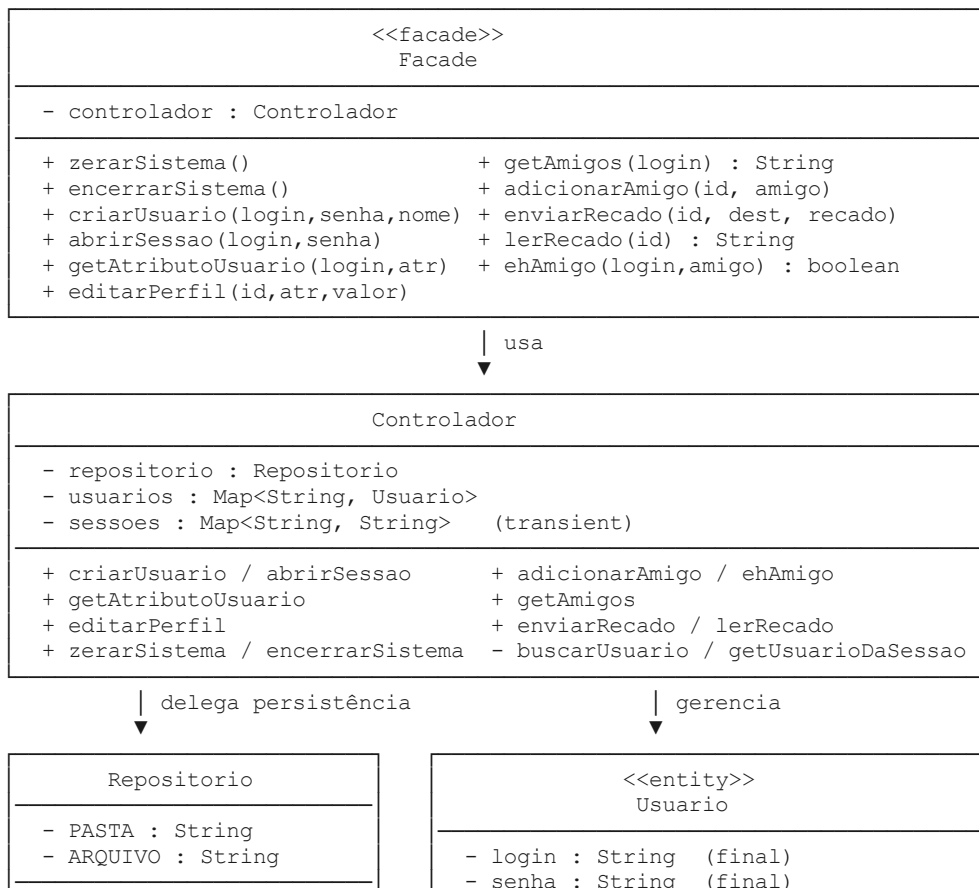
- Controlador — orquestra o fluxo, gerencia sessões e aplica regras de negócio;
- Repositorio — responsável exclusivamente pela persistência em disco;
- Usuario (entidade) — modelo rico que garante a própria integridade;
- Pacote exceptions — exceções atômicas para cada violação de regra.

3.2 Estrutura de Pacotes

Pacote / Arquivo	Responsabilidade
Facade.java	Ponto de entrada do EasyAccept. Despachante puro sem lógica.
Controlador.java	Orquestra entidades, sessões e regras de fluxo.
Repositorio.java	Serialização/desserialização do estado em data/dados.dat.
entities/Usuario.java	Entidade rica: valida estado, encapsula amizades e recados.
exceptions/*.java	12 exceções atômicas herdando de JackutException.

4. Diagrama de Classes

O diagrama a seguir apresenta as principais classes, seus atributos, métodos e relacionamentos:



```
+ carregar()
+ salvar(usuarios)
+ apagar()
```

```
- perfil : Map<String,String>
- amigos : List<String>
- convitesEnviados : Set<String>
- recados : Queue<String>

+ getAtributo / setAtributo
+ verificarSenha(senha) : boolean
+ ehAmigo / temConvitePara : boolean
+ enviarConvite / confirmarAmizade
+ getAmigos() : List (unmodifiable)
+ receberRecado / lerProximoRecado
```

```
<<exceptions>>
JackutException (base) ← LoginInvalidoException
                        ← SenhaInvalidaException
                        ← ContaJaExisteException
                        ← LoginOuSenhaInvalidosException
                        ← UsuarioNaoCadastradoException
                        ← AtributoNaoPreenchidoException
                        ← UsuarioJaAmigoException
                        ← ConvitePendenteException
                        ← AutoAmizadeException
                        ← AutoRecadoException
                        ← SemRecadosException
```

5. Padrões de Projeto Utilizados

5.1 Facade (Fachada)

A classe Facade é o único ponto de contato entre o framework de testes EasyAccept e o sistema. Ela não contém nenhuma lógica de negócio, validação ou instanciação direta de modelos: atua exclusivamente como despachante (delegator), repassando chamadas ao Controlador.

Justificativa: O padrão Facade isola a complexidade interna do sistema de quem o consome. Se a estrutura interna mudar, o contrato público da Facade permanece estável. Isso é especialmente relevante no contexto do EasyAccept, que é extremamente sensível à assinatura dos métodos.

5.2 Rich Domain Model (Modelo de Domínio Rico)

A entidade Usuario não é um simples repositório de dados (Anemic Domain Model). Ela é responsável por garantir a própria integridade:

- Valida login e senha no construtor, lançando LoginInvalidoException e SenhaInvalidaException antes mesmo de o objeto existir com estado inválido;
- Encapsula amizades e recados em coleções internas privadas, expondo apenas operações semânticas (confirmarAmizade, enviarConvite, lerProximoRecado);
- Retorna cópias não-modificáveis das coleções via Collections.unmodifiableList, protegendo o estado interno;
- Lança SemRecadosException diretamente ao tentar ler fila vazia, sem expor isEmpty() para o Controlador.

Justificativa: Concentrar as regras de estado no próprio modelo reduz o acoplamento, evita que o Controlador precise inspecionar o interior das entidades e torna o código autodocumentado.

5.3 Separação de Responsabilidades (SRP — SOLID)

O Controlador original acumulava responsabilidades de negócio e de persistência. A refatoração extraiu a persistência para a classe Repositorio, resultando em dois motivos de mudança independentes:

- Repositorio muda se a tecnologia de armazenamento mudar (ex: trocar serialização por JSON ou banco de dados);
- Controlador muda se as regras de negócio ou o fluxo mudarem.

Justificativa: Ao separar essas responsabilidades, cada classe pode evoluir de forma independente sem afetar a outra. O Controlador não possui mais nenhum import de java.io.

5.4 Hierarquia de Exceções Atômicas

Cada violação de regra de negócio possui sua própria classe de exceção, todas herdando de JackutException (que estende RuntimeException):

Exceção	Situação que a origina
LoginInvalidoException	Login nulo ou vazio ao criar usuário
SenhaInvalidaException	Senha nula ou vazia ao criar usuário
ContaJaExisteException	Login já cadastrado no sistema
LoginOuSenhaInvalidosException	Credenciais incorretas ao abrir sessão
UsuarioNaoCadastradoException	Login ou sessão não encontrados
AtributoNaoPreenchidoException	Atributo de perfil não definido
UsuarioJaAmigoException	Amizade já confirmada entre os dois
ConvitePendenteException	Convite já enviado, aguardando aceite
AutoAmizadeException	Usuário tentando adicionar a si mesmo
AutoRecadoException	Usuário tentando enviar recado a si mesmo
SemRecadosException	Fila de recados vazia ao tentar ler

Justificativa: Exceções atômicas eliminam strings duplicadas no código, tornam cada caso de erro rastreável individualmente e garantem que as mensagens exigidas pelo EasyAccept nunca sejam escritas em mais de um lugar.

6. Decisões Técnicas Relevantes

6.1 Persistência por Serialização

O estado do sistema é persistido em `data/dados.dat` via serialização Java (`ObjectOutputStream/ObjectInputStream`). A pasta `data/` é criada automaticamente pelo Repositorio na primeira execução. Sessões são transient e não persistidas — elas são voláteis por natureza.

O encerramento do sistema (`encerrarSistema`) salva o mapa de usuários e o carregamento acontece automaticamente no construtor do Controlador. O `zerarSistema` limpa memória e apaga o arquivo.

6.2 Sistema de Amizades Bilateral

A amizade no Jackut funciona por convite: usuário A envia convite para B (`convitesEnviados`), e a amizade só é confirmada quando B adiciona A de volta. Nesse momento, ambos os lados chamam `confirmarAmizade()`, que remove o convite pendente e insere o login na lista de amigos.

A lista de amigos preserva a ordem de inserção (`ArrayList`) para garantir o formato de saída determinístico exigido pelos testes de aceitação.

6.3 Fila de Recados FIFO

Os recados são armazenados em uma `Queue<String>` (`LinkedList`) por destinatário. A leitura remove o primeiro elemento (`poll`), garantindo ordem FIFO. A escolha de `String` em vez de uma classe `Recado` é intencional: no escopo atual, o recado não possui estado próprio além do texto. Caso versões futuras exijam remetente, data ou status de leitura, a refatoração para um modelo `Recado` seria natural.

6.4 Execução dos Testes em Cascata

O `EasyAccept` chama `System.exit()` ao encerrar cada script, o que inviabiliza múltiplas chamadas a `EasyAccept.main()` dentro da mesma JVM. A solução adotada foi a classe `Main` lançar cada teste como um subprocesso independente via `ProcessBuilder`, capturando e exibindo a saída no console pai. O arquivo `data/dados.dat` persiste no disco entre subprocessos, garantindo o funcionamento dos testes de persistência (2).

7. Qualidade e Testes

O projeto foi validado com 100% de aprovação nos testes de aceitação do `EasyAccept`, totalizando 184 asserções distribuídas em 8 scripts:

Script	Descrição	Testes	Resultado	Tipo
us1_1	Criação de conta e sessão	17	✓ OK	Funcional
us1_2	Persistência de contas	7	✓ OK	Persistência
us2_1	Edição de perfil	36	✓ OK	Funcional
us2_2	Persistência de perfil	13	✓ OK	Persistência

us3_1	Sistema de amizades	46	✓ OK	Funcional
us3_2	Persistência de amizades	10	✓ OK	Persistência
us4_1	Envio e leitura de recados	42	✓ OK	Funcional
us4_2	Persistência de recados	13	✓ OK	Persistência
Total		184	184 ✓	

8. Conclusão

O design do Jackut foi construído com foco em coesão, encapsulamento e clareza de responsabilidades. O padrão Facade garante um contrato estável com o framework de testes. O Rich Domain Model concentra as regras de integridade nas entidades, reduzindo o acoplamento. A hierarquia de exceções atômicas torna cada caso de erro rastreável e sem duplicação de mensagens. A separação do Repositorio aplica o Princípio da Responsabilidade Única de forma concreta e pragmática.

O resultado é um sistema que compila, passa 100% dos testes de aceitação e apresenta uma arquitetura que pode crescer de forma organizada nos milestones seguintes, bastando adicionar novas entidades, novos métodos na Facade e novas exceções atômicas sem alterar o que já existe.